APLICACIÓN DE PATRONES DE DISEÑO GRASP Y GOF EN UN SISTEMA DE INFORMACIÓN ACADÉMICO

PRESENTADO POR:
ALEJANDRO DE MENDOZA

POLITECNICO GRANCOLOMBIANO
MAESTRÍA ARQUITECTURA DE SOFTWARE
TEMAS AVANZADOS DE DISEÑO SOFTWARE

PRESENTADO AL PROFESOR:
ING. LINA MARIA MONTOYA SUAREZ

15 DE ABRIL
BOGOTÁ D.C.
2026

TABLA DE CONTENIDO

INTRODUCCIÓN ____ *4*

DESARROLLO DEL PROYECTO ____ *4*

Parte 1. Descripción del Problema ____ **5**
Contexto del Sistema ____ 6
El Problema: Generación de Reportes Académicos ____ 6
Manifestación del Problema en el Código ____ 7
Sintomas identificados en el codigo actual ____ 8
Por qué es un Problema de Diseño ____ 9

Parte 2. Patrones de Diseño Identificados ____ **10**
Patron GRASP: Alta Cohesion y Bajo Acoplamiento ____ 11
Patron GoF: Strategy (Estrategia) ____ 12
Definición formal del patron Strategy (GoF) ____ 13
Justificación de la Elección del Patron Strategy ____ 13

Parte 3: Propuesta de Diseño ____ **14**
Arquitectura General de la Solución ____ 15
1. Capa de Datos y Dominio ____ 15
2. Capa de Lógica de Negocio ____ 15
3. Capa de Presentación / Generación de Reportes ____ 16
Beneficios de la Arquitectura Propuesta ____ 16
Participantes del Patrón Strategy en el SIA ____ 16
Strategy (Interfaz) - ReporteEstrategia ____ 16
Concrete Strategies (Estrategias Concretas) ____ 17
Context (Contexto) - ReporteAcademicoService (Refactorizado) ____ 17
Client (Cliente) - ReporteController ____ 18
Valor Arquitectónico de esta Estructura ____ 18
Pseudocódigo de Implementación en Java ____ 18
Interfaz Strategy - ReporteEstrategia.java ____ 19
Descripción de los métodos ____ 19
Justificación del diseño ____ 19
Relación con los principios de diseño ____ 20
Concrete Strategy - ReportePDFEstrategia.java ____ 20
Descripción del componente ____ 21
Responsabilidad y enfoque de diseño ____ 21
Uso de dependencias externas ____ 21
Métodos implementados ____ 21
Integración con el patrón Strategy ____ 22
Beneficios de esta implementación ____ 22
Context refactorizado - ReporteAcademicoService.java ____ 22
Rol de la clase dentro del patrón Strategy ____ 23
Responsabilidades de la clase refactorizada ____ 23
Aplicación de GRASP en esta clase ____ 24
Relación con principios SOLID ____ 24
Ventajas del contexto refactorizado ____ 24
Valor arquitectónico de esta refactorización ____ 24
Client - ReporteController.java ____ 24

Rol de la clase dentro del patrón Strategy 25
Responsabilidades del controlador 25
Ventajas de esta implementación 26
Relación con principios de diseño 26
Valor arquitectónico del cliente en la solución 27

Parte 4: Comparativa de Impacto 27
Antes y Después de Aplicar los Patrones 27
Análisis de Resultados 28
Métricas de Calidad de Software 29
Reducción del Fan-Out (Dependencias Salientes) 29
Mejora en la Inestabilidad del Módulo 29
Impacto en la Abstracción (A) 30
Relación entre Métricas y Calidad del Software 30
Conclusión del Análisis Métrico 30

Parte 5: Beneficios Adicionales del Diseño 30
Paralelismo y Procesamiento Concurrente 30
Inversión de Control (IoC) y Configuración Flexible 31
Extensibilidad mediante combinación con otros patrones (Decorator) 31
Facilidad para Pruebas A/B 31
Auditoría y Trazabilidad 31
Escalabilidad y Evolución del Sistema 32

Parte 6: Diagrama UML 32
Imagen del Diagrama UML 32
Código del Diagrama UML 33
Código y Diagrama UML en Visual Studio Code 35

CONCLUSIONES DEL DESARROLLO 36

BIBLIOGRAFÍA 37

AGRADECIMIENTO 38

INTRODUCCIÓN

En el contexto actual del desarrollo de software, el modelado y diseño de sistemas se han consolidado como etapas fundamentales para garantizar soluciones estructuradas, escalables y de alta calidad. Más allá de la simple implementación funcional, es necesario definir arquitecturas sólidas que permitan comprender claramente los componentes del sistema, sus responsabilidades y las interacciones entre ellos. En este sentido, el uso de enfoques como el Lenguaje Unificado de Modelado (UML) y la aplicación de patrones de diseño facilitan la construcción de sistemas robustos y mantenibles.

El paradigma orientado a objetos juega un papel clave en este proceso, ya que permite identificar las entidades principales del dominio, definir sus atributos y comportamientos, y establecer relaciones coherentes entre ellas. Este enfoque promueve principios fundamentales como la modularidad, el bajo acoplamiento y la alta cohesión, los cuales son esenciales para reducir la complejidad del software y facilitar su evolución en el tiempo.

A partir de estos fundamentos, el presente trabajo aborda el análisis y rediseño de un módulo crítico dentro de un Sistema de Información Académico (SIA), específicamente el proceso de generación de reportes académicos. Este módulo, debido a su crecimiento orgánico, presenta problemas significativos de diseño relacionados con la acumulación de responsabilidades, alto acoplamiento y baja mantenibilidad, afectando directamente la calidad del software y la eficiencia del equipo de desarrollo.

Con el fin de dar solución a esta problemática, se propone la aplicación de patrones de diseño tanto desde la perspectiva de GRASP (asignación de responsabilidades) como de GoF (estructuración de soluciones). En particular, se emplean los principios de **Alta Cohesión** e **Information Expert** para redistribuir adecuadamente las responsabilidades del sistema, junto con el patrón **Strategy**, que permite encapsular los distintos algoritmos de generación de reportes (PDF, Excel, HTML, entre otros) de forma independiente y extensible.

Finalmente, a través de este enfoque, se busca no solo mejorar la estructura del sistema, sino también reducir la cantidad de errores, facilitar las pruebas, disminuir el tiempo de desarrollo ante nuevos requerimientos y asegurar una arquitectura alineada con principios de ingeniería de software moderna. De esta manera, el documento presenta un análisis detallado del problema, la justificación de los patrones seleccionados y una propuesta de diseño que optimiza la calidad y mantenibilidad del sistema.

DESARROLLO DEL PROYECTO

A continuación, se presenta el desarrollo estructurado de la actividad orientada a la aplicación de patrones de diseño en un sistema de información, con el propósito de analizar un problema real de software y proponer una solución basada en buenas prácticas de ingeniería. El objetivo principal de este ejercicio es aplicar los fundamentos del diseño orientado a objetos junto con los patrones GRASP y GoF, con el fin de construir una solución que mejore la calidad, mantenibilidad y escalabilidad del sistema.

El desarrollo se organiza en diferentes secciones que abordan de manera progresiva el análisis del problema y la propuesta de solución. En una primera etapa, se describe detalladamente el problema identificado dentro del Sistema de Información Académico (SIA),

justificando por qué representa una deficiencia de diseño y cómo impacta negativamente en la evolución del software. Posteriormente, se realiza un análisis de los principios y patrones de diseño aplicables, destacando la importancia de una correcta asignación de responsabilidades mediante GRASP y el uso de estructuras flexibles mediante patrones GoF.

A partir de este análisis, se propone un rediseño del sistema enfocado en el módulo de generación de reportes académicos, el cual presenta problemas de alta complejidad, bajo nivel de cohesión y fuerte acoplamiento. La solución planteada incorpora principios como la separación de responsabilidades, el uso del polimorfismo y la encapsulación de comportamientos variables, permitiendo transformar un diseño monolítico en una arquitectura modular y extensible.

Asimismo, se describe la estructura del sistema resultante, incluyendo la definición de las clases principales, sus responsabilidades, así como las relaciones que existen entre ellas. Se hace especial énfasis en la implementación del patrón Strategy, el cual permite gestionar múltiples formatos de generación de reportes de manera independiente, y en la aplicación de los principios GRASP para garantizar una distribución adecuada de responsabilidades dentro del sistema.

En las siguientes secciones se detalla la propuesta de diseño, incluyendo su representación conceptual y su implementación a nivel de pseudocódigo, permitiendo visualizar de manera clara la solución planteada. Este enfoque no solo facilita la comprensión del sistema, sino que también evidencia cómo la aplicación de patrones de diseño contribuye directamente a la reducción de errores, la mejora en la calidad del software y la optimización del proceso de desarrollo.

Parte 1. Descripción Del Problema

Una vez establecidos los fundamentos teóricos del diseño orientado a objetos y la importancia de aplicar patrones de diseño en el desarrollo de software, resulta fundamental analizar situaciones reales en las cuales la ausencia de estos principios genera problemas significativos en la calidad del sistema.

En los sistemas de información empresariales, es común que, debido al crecimiento progresivo y a la incorporación constante de nuevos requerimientos, ciertas funcionalidades evolucionen sin una adecuada estructura de diseño. Esta situación conduce a la aparición de código altamente acoplado, difícil de mantener y propenso a errores, lo que afecta directamente la estabilidad y escalabilidad del sistema.

En este contexto, el módulo de generación de reportes dentro de un Sistema de Información Académico (SIA) representa un caso típico donde la falta de una adecuada asignación de responsabilidades y la ausencia de patrones de diseño generan problemas estructurales importantes. Dicho módulo, al concentrar múltiples funcionalidades en una sola clase, evidencia deficiencias que impactan tanto en el desarrollo como en el mantenimiento del software.

A continuación, se presenta una descripción detallada del problema identificado, analizando sus causas, su manifestación en el código y las consecuencias que tiene sobre la

calidad del sistema, con el fin de establecer una base sólida para la posterior aplicación de patrones de diseño que permitan su solución.

Contexto Del Sistema

El Sistema de Información Académico (SIA) objeto de estudio corresponde a una plataforma web institucional de misión crítica, utilizada por una universidad de tamaño medio con aproximadamente 8.000 estudiantes activos, 400 docentes y cerca de 60 programas académicos distribuidos en diferentes facultades y niveles de formación.

Este sistema constituye el núcleo operativo de la gestión académica institucional, ya que soporta de manera integral el ciclo de vida del estudiante dentro de la universidad. Entre sus principales funcionalidades se incluyen la inscripción y cancelación de asignaturas, la gestión de horarios, el registro y control de asistencias, la carga y consulta de calificaciones, la generación de certificados académicos y la producción de reportes de rendimiento tanto individuales como agregados.

Adicionalmente, el SIA se integra con otros sistemas institucionales, como el sistema financiero para la validación de pagos de matrícula, plataformas de bienestar universitario para el seguimiento de estudiantes, y servicios externos que requieren intercambio de información estructurada. Esta integración implica el manejo de múltiples fuentes de datos y la necesidad de garantizar consistencia, disponibilidad y seguridad en la información procesada.

Desde una perspectiva técnica, el sistema presenta características propias de una aplicación empresarial compleja: múltiples actores con distintos roles (estudiantes, docentes, coordinadores y administrativos), alta concurrencia de usuarios en periodos críticos (como matrículas o cierre de notas), y una gran diversidad de requerimientos funcionales y no funcionales. Estas condiciones demandan una arquitectura bien definida que permita escalar, mantener y evolucionar el sistema sin comprometer su estabilidad.

En este contexto, cualquier deficiencia en el diseño del software, especialmente en módulos críticos como el de generación de reportes académicos, puede tener un impacto significativo en la operación institucional. Por esta razón, resulta fundamental analizar en detalle la estructura interna del sistema, identificar problemas de diseño y aplicar patrones adecuados que permitan mejorar su calidad, reducir errores y facilitar su evolución a largo plazo.

El Problema: Generación De Reportes Académicos

Uno de los módulos más críticos y, a la vez, más problemáticos dentro del Sistema de Información Académico (SIA) es el módulo de **Generación de Reportes Académicos**. Este componente cumple un rol fundamental dentro del sistema, ya que es el encargado de transformar grandes volúmenes de datos académicos en información estructurada, comprensible y útil para la toma de decisiones.

Dicho módulo tiene la responsabilidad de generar documentos en múltiples formatos, tales como PDF, Excel, HTML, CSV y Word, los cuales son utilizados por diferentes tipos de usuarios y sistemas. Por ejemplo, los estudiantes requieren reportes como su historial académico o certificados de notas; los docentes necesitan listados de calificaciones y asistencia;

los coordinadores académicos analizan reportes estadísticos para la toma de decisiones; y, adicionalmente, existen sistemas externos que consumen esta información en formatos estructurados para procesos administrativos o de integración institucional.

Esta diversidad de formatos y de consumidores implica que el módulo no solo debe presentar la información, sino también adaptarla a diferentes contextos, estructuras y requerimientos técnicos. Cada formato posee particularidades específicas en cuanto a diseño, estructura de datos, librerías utilizadas y forma de presentación, lo que introduce una variabilidad significativa en el comportamiento del sistema.

Asimismo, la generación de reportes no es un proceso trivial, ya que involucra múltiples etapas como la obtención de datos desde distintas fuentes, el procesamiento de información académica (por ejemplo, cálculo de promedios, estados académicos o indicadores de rendimiento), y finalmente su transformación en un formato específico de salida. Esta combinación de responsabilidades convierte al módulo en un punto de alta complejidad dentro del sistema.

En consecuencia, cuando este tipo de funcionalidad no es diseñada adecuadamente, tiende a convertirse en un componente altamente acoplado, difícil de mantener y propenso a errores. La necesidad de soportar nuevos formatos, cambios en los requisitos o ajustes en la lógica de negocio incrementa el riesgo de introducir fallos en funcionalidades existentes, afectando la estabilidad del sistema.

Por lo tanto, el módulo de generación de reportes académicos representa un caso claro donde la falta de una adecuada separación de responsabilidades y la ausencia de patrones de diseño apropiados generan problemas estructurales que impactan directamente en la calidad del software, justificando la necesidad de un análisis profundo y la aplicación de soluciones basadas en buenas prácticas de diseño.

Manifestación Del Problema En El Código

En la implementación actual del sistema, el problema descrito se materializa principalmente en una clase denominada ReporteAcademicoService, la cual ha evolucionado de manera orgánica a lo largo de varios años como resultado de la incorporación continua de nuevos requerimientos sin una adecuada refactorización o rediseño estructural.

Esta clase concentra una gran cantidad de responsabilidades heterogéneas que abarcan diferentes capas del sistema. Entre sus funciones principales se encuentran: la obtención de datos del estudiante desde la base de datos mediante repositorios, el procesamiento de información académica como el cálculo de promedios y estadísticas, la aplicación de reglas de negocio (por ejemplo, la determinación de estados académicos como “en riesgo” o “aprobado”), así como la generación de reportes en múltiples formatos utilizando diferentes librerías externas como iText para PDF, Apache POI para Excel y motores de plantillas para HTML. Adicionalmente, la misma clase gestiona el envío de los reportes generados a través de correo electrónico.

Esta acumulación de responsabilidades convierte a ReporteAcademicoService en un claro ejemplo de una **“God Class” (clase dios)**, caracterizada por su alto nivel de complejidad,

baja cohesión y fuerte acoplamiento con múltiples dependencias externas. Como consecuencia, el código presenta múltiples problemas estructurales:

En primer lugar, existe una fuerte duplicación de lógica entre los distintos métodos de generación de reportes (por ejemplo, generarReportePDF(), generarReporteExcel() y generarReporteHTML()), los cuales comparten gran parte del flujo de procesamiento de datos, diferenciándose únicamente en la forma de presentación. Esta duplicidad incrementa el esfuerzo de mantenimiento y eleva el riesgo de inconsistencias.

En segundo lugar, la clase presenta un alto nivel de acoplamiento con librerías específicas, lo que dificulta la sustitución o actualización de tecnologías sin afectar directamente el núcleo del sistema. Cualquier cambio en una librería de generación de documentos puede impactar múltiples partes del código, generando efectos colaterales no deseados.

En tercer lugar, la ausencia de separación de responsabilidades impide la correcta aplicación de pruebas unitarias. Dado que la clase combina acceso a datos, lógica de negocio y generación de documentos, resulta complejo aislar funcionalidades específicas para su validación, lo que reduce significativamente la cobertura de pruebas y aumenta la probabilidad de errores en producción.

Adicionalmente, el crecimiento descontrolado de la clase ha generado un aumento considerable en su tamaño y complejidad, dificultando su comprensión por parte de nuevos desarrolladores y elevando el tiempo necesario para realizar modificaciones. Esto se traduce en una disminución de la productividad del equipo y un mayor riesgo de introducir defectos durante el mantenimiento.

Finalmente, cualquier cambio o extensión del sistema, como la incorporación de un nuevo formato de reporte, requiere modificar directamente esta clase, lo que incrementa el riesgo de regresión en funcionalidades existentes. Esta situación evidencia claramente la falta de aplicación de principios fundamentales de diseño y patrones que permitan desacoplar y modularizar el comportamiento del sistema.

En conjunto, estos problemas reflejan una deficiencia estructural en el diseño del software, justificando la necesidad de aplicar patrones de diseño que permitan redistribuir responsabilidades, reducir el acoplamiento y mejorar la calidad general del sistema.

Síntomas Identificados En El Código Actual

El análisis del código actual del módulo de generación de reportes permite identificar una serie de síntomas claros que evidencian problemas estructurales en el diseño del sistema. Estos síntomas no solo afectan la calidad del código, sino que también tienen un impacto directo en la mantenibilidad, escalabilidad y confiabilidad del software.

En primer lugar, la clase ReporteAcademicoService presenta un tamaño excesivo, superando las 2.800 líneas de código, lo cual es un indicador evidente de una mala distribución de responsabilidades. Este volumen de código dificulta su comprensión, incrementa la complejidad cognitiva y eleva significativamente el riesgo de errores durante su modificación.

En segundo lugar, se observa una alta duplicación de lógica en métodos como generarReportePDF(), generarReporteExcel() y generarReporteHTML(). Aunque cada uno genera un formato distinto, comparten gran parte del flujo de procesamiento de datos, lo que implica que cualquier cambio en la lógica base debe replicarse manualmente en múltiples lugares, aumentando el riesgo de inconsistencias y defectos.

Otro síntoma relevante es que cada vez que se requiere agregar un nuevo formato de reporte, es necesario modificar directamente la clase principal. Este comportamiento evidencia una violación del principio de abierto/cerrado (OCP), ya que el sistema no está diseñado para extenderse sin alterar código existente, lo que incrementa la probabilidad de introducir errores en funcionalidades previamente estables.

Adicionalmente, el diseño actual dificulta la realización de pruebas unitarias. Debido a la mezcla de responsabilidades -acceso a datos, lógica de negocio y generación de documentos- no es posible aislar componentes específicos para su validación, lo que limita la cobertura de pruebas y reduce la capacidad de detectar fallos de manera temprana.

Desde el punto de vista de construcción y despliegue, se evidencia también una ineficiencia operativa, ya que cualquier cambio relacionado con un formato específico obliga a recompilar todo el módulo. Esto impacta negativamente en los tiempos de desarrollo y en la agilidad del equipo.

Finalmente, uno de los problemas más críticos es el acoplamiento interno entre funcionalidades. Se han identificado casos en los cuales errores en el generador de un formato (por ejemplo, PDF) afectan indirectamente la generación de otros formatos como Excel. Esto ocurre debido al uso compartido de estado interno y a la falta de encapsulamiento adecuado, lo que genera efectos colaterales no deseados.

En conjunto, estos síntomas reflejan un diseño frágil y poco flexible, que dificulta la evolución del sistema y aumenta el riesgo de fallos en producción. Esta situación justifica la necesidad de aplicar patrones de diseño que permitan mejorar la estructura del código, reducir la duplicación, desacoplar responsabilidades y aumentar la calidad general del software.

Por Qué Es Un Problema De Diseño

El escenario descrito no corresponde únicamente a un problema de implementación, sino a una deficiencia estructural en el diseño del software. La acumulación de responsabilidades en una única clase y la falta de una arquitectura adecuada evidencian la ausencia de principios fundamentales de la ingeniería de software, lo cual impacta directamente en la calidad, mantenibilidad y evolución del sistema.

En primer lugar, se identifica una clara violación del **Principio de Responsabilidad Única (SRP)**, el cual establece que una clase debe tener una única razón para cambiar. En el caso analizado, la clase ReporteAcademicoService asume múltiples responsabilidades: acceso a datos, procesamiento de información académica, aplicación de reglas de negocio y generación de reportes en diversos formatos. Esta sobrecarga de funciones genera un alto acoplamiento interno y dificulta la comprensión, prueba y mantenimiento del código.

En segundo lugar, se incumple el **Principio Abierto/Cerrado (OCP)**, que indica que las entidades de software deben estar abiertas a extensión pero cerradas a modificación. En el diseño actual, cada vez que se requiere incorporar un nuevo formato de reporte -por ejemplo, JSON para integración con servicios REST- es necesario modificar directamente la clase existente. Esto no solo incrementa el riesgo de introducir errores, sino que también limita la capacidad del sistema para evolucionar de manera controlada.

Asimismo, se evidencia la ausencia del **Principio de Sustitución de Liskov (LSP)**, ya que no existe una jerarquía de abstracción que permita intercambiar diferentes implementaciones de generación de reportes mediante polimorfismo. La falta de interfaces o clases abstractas impide desacoplar el comportamiento y reutilizar código de manera eficiente.

Desde la perspectiva de los patrones GRASP, el sistema presenta una violación directa de los principios de **Alta Cohesión** y **Bajo Acoplamiento**. La clase analizada posee una cohesión funcional muy baja, dado que agrupa responsabilidades no relacionadas, y un alto nivel de acoplamiento debido a su dependencia directa con múltiples librerías externas y componentes del sistema. Esta situación dificulta la reutilización de código y aumenta la fragilidad del sistema ante cualquier cambio.

El impacto de estas deficiencias de diseño se refleja en problemas concretos dentro del proyecto. En términos de mantenimiento, el tiempo requerido para realizar modificaciones se ha incrementado considerablemente, llegando incluso a triplicarse en los últimos años. Además, el equipo de desarrollo muestra resistencia a intervenir el módulo de reportes debido al alto riesgo de regresión, lo cual es un síntoma claro de un sistema poco confiable.

En cuanto a la calidad del software, la cobertura de pruebas automatizadas es inferior al 15%, lo que limita la capacidad de detectar errores de forma temprana. Adicionalmente, la incorporación de un nuevo formato de reporte puede tardar en promedio hasta tres semanas, debido a la necesidad de realizar múltiples ajustes y pruebas manuales para evitar impactos en funcionalidades existentes.

En conjunto, estos factores demuestran que el problema no radica únicamente en la complejidad del módulo, sino en un diseño inadecuado que no sigue principios ni patrones establecidos. Por esta razón, resulta imprescindible aplicar enfoques de diseño como los patrones GRASP y GoF, que permitan reorganizar las responsabilidades, reducir el acoplamiento y construir una arquitectura más robusta, flexible y mantenible.

Parte 2. Patrones De Diseño Identificados

Una vez analizado el problema y evidenciadas las deficiencias estructurales en el diseño del módulo de generación de reportes, resulta necesario plantear una solución fundamentada en buenas prácticas de ingeniería de software. En este contexto, los patrones de diseño se presentan como una herramienta clave para abordar problemas recurrentes de manera estructurada, reutilizable y comprobada.

Los patrones de diseño permiten no solo mejorar la organización del código, sino también facilitar la comprensión del sistema, reducir el acoplamiento entre componentes y aumentar la cohesión de las clases. De esta manera, contribuyen directamente a la construcción de software más mantenible, extensible y menos propenso a errores.

Para dar solución al problema identificado, se realizará un análisis desde dos perspectivas complementarias. Por un lado, se aplicarán los patrones GRASP (General Responsibility Assignment Software Patterns), los cuales orientan la correcta asignación de responsabilidades entre las clases del sistema. Por otro lado, se emplearán patrones de la familia GoF (Gang of Four), que proporcionan estructuras de solución probadas para resolver problemas específicos de diseño.

En particular, se identificarán y justificarán aquellos patrones que mejor se ajustan al contexto del sistema, permitiendo transformar un diseño monolítico y altamente acoplado en una arquitectura modular, flexible y orientada a la extensibilidad. Esta combinación de enfoques no solo permitirá resolver el problema actual, sino también prevenir la aparición de problemas similares en futuras evoluciones del sistema.

A continuación, se presentan los patrones de diseño seleccionados, junto con su justificación y la forma en que contribuyen a mejorar la calidad del software dentro del Sistema de Información Académico.

Patrón GRASP: Alta Cohesión Y Bajo Acoplamiento

Dentro de los nueve patrones GRASP (General Responsibility Assignment Software Patterns) propuestos por Craig Larman en su obra Applying UML and Patterns, el problema identificado en el módulo de generación de reportes del SIA evidencia una violación directa de los principios de Alta Cohesión (High Cohesion) y Experto en Información (Information Expert), los cuales son fundamentales para lograr un diseño orientado a objetos correcto.

El principio de Alta Cohesión establece que una clase debe tener responsabilidades claramente definidas y estrechamente relacionadas entre sí, evitando la acumulación de funcionalidades heterogéneas. Una clase altamente cohesiva es más fácil de comprender, mantener, reutilizar y probar, ya que su propósito dentro del sistema es claro y bien delimitado.

En el caso analizado, la clase ReporteAcademicoService presenta una cohesión funcional muy baja, dado que agrupa múltiples responsabilidades que pertenecen a diferentes niveles del sistema, tales como el acceso a datos, el procesamiento de información académica, la aplicación de reglas de negocio y la generación de reportes en distintos formatos. Esta mezcla de responsabilidades genera una estructura rígida, difícil de mantener y altamente propensa a errores.

Por otro lado, el principio de Bajo Acoplamiento, estrechamente relacionado con la cohesión, busca minimizar las dependencias entre clases, de manera que los cambios en un componente tengan el menor impacto posible sobre los demás. En el diseño actual, la clase ReporteAcademicoService presenta un alto nivel de acoplamiento, ya que depende directamente de múltiples librerías externas (como iText y Apache POI) y de distintos componentes del sistema, lo que dificulta su evolución y aumenta el riesgo de efectos colaterales ante cualquier modificación.

Como complemento a estos principios, el patrón GRASP de Experto en Información (Information Expert) proporciona una guía clara para la asignación adecuada de responsabilidades. Este patrón establece que una responsabilidad debe asignarse a la clase que posee la información necesaria para cumplirla. Aplicado al contexto del SIA, esto implica que

las operaciones relacionadas con el cálculo de promedios deben ser responsabilidad de la clase Estudiante, ya que esta contiene los datos necesarios (calificaciones), mientras que la generación de documentos en formatos específicos debe delegarse a clases especializadas que conozcan exclusivamente la lógica de presentación.

A partir de estos principios, la solución propuesta consiste en redistribuir las responsabilidades del sistema, separando claramente las funciones de obtención de datos, procesamiento de información y generación de reportes. De esta manera, cada clase asume un rol específico dentro del sistema, lo que permite aumentar la cohesión, reducir el acoplamiento y mejorar significativamente la calidad del diseño.

En consecuencia, la aplicación de los patrones GRASP no solo corrige las deficiencias estructurales identificadas, sino que también establece una base sólida para la incorporación de otros patrones de diseño, como Strategy, que permiten gestionar la variabilidad del sistema de manera flexible y extensible.

Patrón GoF: Strategy (Estrategia)

De los 23 patrones de diseño documentados en la obra *Design Patterns: Elements of Reusable Object-Oriented Software* (Gamma, Helm, Johnson y Vlissides, 1994), el patrón que mejor se ajusta para resolver la problemática identificada en el módulo de generación de reportes es el **patrón Strategy**, el cual pertenece a la categoría de patrones de comportamiento (*Behavioral Patterns*).

El patrón Strategy tiene como propósito principal definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables dentro de un mismo contexto. De esta manera, permite que el comportamiento de un sistema pueda variar dinámicamente sin necesidad de modificar el código cliente que lo utiliza.

En el contexto del Sistema de Información Académico (SIA), la generación de reportes en distintos formatos (PDF, Excel, HTML, CSV, entre otros) puede interpretarse como un conjunto de algoritmos que comparten un mismo objetivo -generar un reporte académico- pero difieren en su implementación. Cada formato representa una variación del mismo proceso, con reglas específicas de construcción, presentación y uso de librerías externas.

En el diseño actual, esta variabilidad está implementada de forma rígida dentro de una única clase, lo que genera duplicación de código, alto acoplamiento y dificultad para extender el sistema. El patrón Strategy permite resolver este problema al separar cada uno de estos comportamientos en clases independientes que implementan una interfaz común, desacoplando así la lógica de generación de reportes del resto del sistema.

De esta forma, el sistema puede seleccionar en tiempo de ejecución la estrategia más adecuada según el formato requerido, sin necesidad de utilizar estructuras condicionales complejas ni modificar el código existente. Esto no solo mejora la flexibilidad del sistema, sino que también facilita la incorporación de nuevos formatos en el futuro, cumpliendo con el principio de abierto/cerrado (OCP).

En consecuencia, la aplicación del patrón Strategy permite transformar un diseño monolítico en una arquitectura modular, donde cada algoritmo de generación de reportes se

encuentra encapsulado, es independiente y puede evolucionar de manera aislada. Esta característica resulta fundamental en sistemas empresariales donde los requerimientos cambian constantemente y la capacidad de adaptación del software es un factor crítico de éxito.

Definición Formal Del Patrón Strategy (GoF)

El patrón de diseño Strategy, perteneciente al catálogo de patrones de comportamiento (Behavioral Patterns) de la GoF, se define como un mecanismo que permite definir una familia de algoritmos, encapsular cada uno de ellos en clases independientes y hacerlos intercambiables dentro de un mismo contexto.

Su objetivo principal es separar el comportamiento variable de un sistema de aquellos componentes que lo utilizan, permitiendo que dichos comportamientos puedan modificarse, extenderse o sustituirse sin afectar al resto de la aplicación. En este sentido, Strategy promueve el uso de la abstracción mediante interfaces o clases abstractas, las cuales definen un contrato común que todas las implementaciones concretas deben cumplir.

Desde una perspectiva estructural, el patrón se compone de tres elementos fundamentales:

- **Strategy (Interfaz o abstracción):** Define el contrato común para todos los algoritmos.
- **Concrete Strategies (Implementaciones concretas):** Representan las diferentes variantes del algoritmo, cada una encapsulada en su propia clase.
- **Context (Contexto):** Es la clase que utiliza una estrategia y delega en ella la ejecución del comportamiento, sin conocer los detalles de su implementación.

Una de las características más relevantes de este patrón es que permite seleccionar la estrategia adecuada en tiempo de ejecución o mediante configuración, lo que proporciona una alta flexibilidad al sistema. Esto elimina la necesidad de utilizar estructuras condicionales complejas (como múltiples if o switch), reemplazándolas por polimorfismo, lo cual mejora la legibilidad y mantenibilidad del código.

En consecuencia, el patrón Strategy facilita la construcción de sistemas abiertos a la extensión, alineándose con principios fundamentales de diseño como el Principio Abierto/Cerrado (OCP) y favoreciendo la reutilización de código, la independencia entre componentes y la reducción del acoplamiento.

Justificación De La Elección Del Patron Strategy

El patrón Strategy se presenta como la solución más adecuada para el problema identificado en el módulo de generación de reportes del Sistema de Información Académico (SIA), ya que permite abordar de manera efectiva la variabilidad del comportamiento del sistema, al mismo tiempo que mejora su estructura, mantenibilidad y capacidad de evolución. A continuación, se detallan las principales razones técnicas y de negocio que justifican su elección:

En primer lugar, se destaca la **variabilidad del algoritmo de generación de reportes**. Los diferentes formatos requeridos (PDF, Excel, HTML, CSV, entre otros) representan distintas

formas de implementar un mismo proceso conceptual: la generación de un reporte académico a partir de un conjunto de datos. Cada uno de estos formatos implica un algoritmo diferente, con reglas específicas de construcción y presentación. El patrón Strategy permite encapsular cada una de estas variantes en clases independientes, tratándolas como algoritmos intercambiables bajo una misma abstracción.

En segundo lugar, el patrón favorece la **independencia de implementación**. Cada estrategia de generación de reportes puede desarrollarse, modificarse y evolucionar de manera aislada, sin afectar a las demás ni al componente que las utiliza. Esto resulta especialmente importante en sistemas empresariales donde los cambios en requerimientos son frecuentes, ya que reduce el impacto de las modificaciones y minimiza el riesgo de introducir errores en otras funcionalidades.

Otro aspecto fundamental es la capacidad de **extensión sin modificación**, alineándose con el Principio Abierto/Cerrado (OCP). En el diseño propuesto, la incorporación de un nuevo formato de reporte no requiere modificar el código existente, sino simplemente crear una nueva clase que implemente la interfaz común. Esto contrasta con el diseño actual, donde cualquier cambio implica intervenir directamente la clase principal, aumentando el riesgo de regresión.

Desde el punto de vista de calidad del software, el patrón Strategy mejora significativamente la **testabilidad del sistema**. Al estar cada algoritmo encapsulado en una clase independiente, es posible realizar pruebas unitarias específicas para cada estrategia utilizando datos controlados. Esto permite incrementar la cobertura de pruebas -que en el sistema actual es inferior al 15%- hasta niveles superiores al 85%, mejorando la confiabilidad del software.

Finalmente, el patrón permite una **selección dinámica del comportamiento**, lo que aporta flexibilidad al sistema. La estrategia de generación de reportes puede determinarse en tiempo de ejecución en función de distintos factores, como el formato solicitado por el usuario, parámetros de configuración o el tipo de cliente que realiza la petición (por ejemplo, web o integración externa). Esta capacidad elimina la necesidad de estructuras condicionales complejas y permite construir un sistema más adaptable y limpio desde el punto de vista arquitectónico.

En conjunto, estas características convierten al patrón Strategy en una solución idónea para el problema planteado, ya que no solo resuelve las deficiencias actuales del sistema, sino que también establece una base sólida para su evolución futura, reduciendo errores, mejorando la mantenibilidad y aumentando la eficiencia del proceso de desarrollo.

Parte 3: Propuesta De Diseño

Una vez analizado el problema y justificada la aplicación de los patrones de diseño más adecuados, resulta necesario plantear una solución estructurada que permita materializar estos principios en una arquitectura concreta. En este sentido, la propuesta de diseño tiene como objetivo transformar el módulo de generación de reportes del Sistema de Información Académico (SIA) en una solución modular, flexible y alineada con buenas prácticas de ingeniería de software.

La propuesta se fundamenta en la correcta asignación de responsabilidades mediante los principios GRASP, así como en la implementación del patrón Strategy de la GoF, con el fin de desacoplar los distintos comportamientos del sistema y facilitar su extensión sin afectar el código existente. De esta manera, se busca eliminar las deficiencias del diseño actual, caracterizado por alta complejidad, bajo nivel de cohesión y fuerte acoplamiento.

A través de esta propuesta, se define una nueva estructura del sistema en la cual cada componente cumple una función específica, permitiendo separar claramente las responsabilidades relacionadas con la obtención de datos, el procesamiento de la información y la generación de reportes. Este enfoque no solo mejora la claridad del diseño, sino que también incrementa la mantenibilidad, la testabilidad y la capacidad de evolución del sistema.

En las siguientes secciones se describirá en detalle la arquitectura propuesta, los componentes involucrados, sus responsabilidades y las relaciones entre ellos, así como una aproximación a su implementación, evidenciando cómo la aplicación de patrones de diseño contribuye directamente a la mejora de la calidad del software.

Arquitectura General De La Solución

La propuesta de diseño planteada integra de manera coherente los principios de asignación de responsabilidades definidos por GRASP con la estructura del patrón Strategy de la GoF, dando como resultado una arquitectura modular, flexible y alineada con buenas prácticas de ingeniería de software.

El objetivo principal de esta arquitectura es transformar el diseño actual -caracterizado por una alta complejidad y fuerte acoplamiento- en una solución desacoplada, donde cada componente tenga una responsabilidad claramente definida y pueda evolucionar de manera independiente.

Para lograr esto, se establece una separación en **tres capas principales de responsabilidad**, cada una con funciones específicas dentro del sistema:

1. Capa De Datos Y Dominio

Esta capa está compuesta por las entidades del negocio, tales como Estudiante, Calificacion y otros objetos del dominio académico. Su responsabilidad principal es representar la información del sistema y encapsular comportamientos directamente relacionados con dichos datos.

Siguiendo el principio GRASP de **Information Expert**, las operaciones que dependen de los datos -como el cálculo del promedio académico- son realizadas por las propias entidades del dominio. Esto permite mantener una alta cohesión y evitar que la lógica de negocio se disperse en otras capas.

2. Capa De Lógica De Negocio

En esta capa se ubica el componente ReporteAcademicoService, pero en su versión refactorizada. Su función principal es coordinar el flujo del proceso de generación de reportes, sin encargarse de los detalles de implementación.

Este servicio es responsable de:

- Obtener la información necesaria desde los repositorios
- Construir objetos de transferencia de datos (DTO)
- Delegar la generación del reporte a la estrategia correspondiente

De esta manera, actúa como un **orquestador**, manteniendo un bajo acoplamiento con las implementaciones concretas gracias al uso de interfaces.

3. Capa De Presentación / Generación De Reportes

Esta capa está compuesta por las diferentes estrategias de generación de reportes (ReportePDFEstrategia, ReporteExcelEstrategia, etc.), las cuales implementan una interfaz común.

Cada una de estas clases encapsula exclusivamente la lógica necesaria para generar un tipo específico de reporte, cumpliendo con el patrón Strategy. Esto permite que cada formato sea independiente, reutilizable y fácilmente extensible.

Beneficios De La Arquitectura Propuesta

La separación en capas y la aplicación de patrones de diseño permiten obtener múltiples beneficios:

- **Desacoplamiento estructural:** cada capa puede evolucionar sin afectar a las demás
- **Alta cohesión:** cada componente cumple una función clara y específica
- **Facilidad de mantenimiento:** los cambios se localizan en componentes aislados
- **Escalabilidad:** el sistema puede crecer sin aumentar su complejidad de forma descontrolada
- **Mejor testabilidad:** cada componente puede ser probado de forma independiente

En conjunto, esta arquitectura no solo resuelve los problemas actuales del sistema, sino que también establece una base sólida para su evolución futura, alineando el diseño con principios modernos de desarrollo de software.

Participantes Del Patrón Strategy En El SIA

La implementación del patrón Strategy en el Sistema de Información Académico (SIA) permite mapear de manera clara los roles definidos por el patrón a componentes concretos dentro del dominio del sistema. Esta correspondencia facilita la comprensión del diseño y evidencia cómo los conceptos teóricos se aplican en una solución real.

A continuación, se describen los principales participantes del patrón y su adaptación al contexto del módulo de generación de reportes:

Strategy (Interfaz) - ReporteEstrategia

Este componente define el contrato común que deben cumplir todas las estrategias de generación de reportes. A través de esta interfaz se abstrae el comportamiento de generación, permitiendo que diferentes implementaciones puedan ser utilizadas de forma intercambiable.

El método principal definido es:

$$byte[\quad]generar(DatosReporteDTO\ datos);$$

Además, pueden incluirse métodos auxiliares como la obtención del tipo de contenido o la extensión del archivo. Esta abstracción es fundamental para lograr desacoplamiento y permitir el uso del polimorfismo dentro del sistema.

Concrete Strategies (Estrategias Concretas)

Cada una de estas clases representa una implementación específica del algoritmo de generación de reportes, encapsulando la lógica particular de cada formato:

- **ReportePDFEstrategia (ConcreteStrategy A):**
  Implementa la generación de reportes en formato PDF utilizando la librería iText 7. Se encarga exclusivamente de la construcción del documento en este formato, sin involucrar lógica de negocio.
- **ReporteExcelEstrategia (ConcreteStrategy B):**
  Genera reportes en formato Excel mediante el uso de Apache POI, estructurando los datos en hojas de cálculo.
- **ReporteHTMLEstrategia (ConcreteStrategy C):**
  Produce representaciones en HTML utilizando motores de plantillas como Thymeleaf, permitiendo su visualización en entornos web.
- **ReporteCSVEstrategia (ConcreteStrategy D):**
  Genera archivos CSV orientados a la exportación de datos estructurados, comúnmente utilizados para integración con otros sistemas.

Cada estrategia encapsula completamente su lógica de generación, lo que permite su evolución independiente y evita la propagación de cambios a otros componentes.

Context (Contexto) - ReporteAcademicoService (Refactorizado)

El contexto es el componente encargado de utilizar una estrategia sin conocer los detalles de su implementación. En este caso, ReporteAcademicoService actúa como un **orquestador del proceso**, manteniendo una referencia a la interfaz ReporteEstrategia.

Sus responsabilidades principales son:

- Obtener los datos necesarios desde el dominio (repositorios)
- Preparar la información en un objeto de transferencia (DatosReporteDTO)
- Delegar la generación del reporte a la estrategia seleccionada

Este enfoque permite que el servicio permanezca desacoplado de las implementaciones concretas, facilitando la mantenibilidad y extensión del sistema.

Client (Cliente) - ReporteController

El cliente es el componente que decide qué estrategia utilizar en función de la solicitud recibida. En este caso, el ReporteController selecciona la estrategia adecuada basándose en el parámetro formato de la petición HTTP.

Posteriormente, esta estrategia es inyectada en el contexto (ReporteAcademicoService), permitiendo que el sistema ejecute dinámicamente el comportamiento requerido.

Este mecanismo de selección dinámica es clave para la flexibilidad del sistema, ya que permite adaptar el comportamiento sin modificar el código existente.

Valor Arquitectónico De Esta Estructura

La correcta identificación e implementación de los participantes del patrón Strategy permite:

- Desacoplar completamente la lógica de generación de reportes
- Eliminar estructuras condicionales complejas
- Facilitar la incorporación de nuevos formatos sin modificar código existente
- Mejorar la claridad y organización del sistema
- Permitir pruebas unitarias independientes por cada estrategia

En conjunto, esta distribución de responsabilidades evidencia una aplicación correcta del patrón Strategy, alineada con los principios de diseño orientado a objetos y orientada a la construcción de software flexible, mantenible y escalable.

Pseudocódigo De Implementación En Java

Una vez definida la arquitectura de la solución y establecidos los participantes del patrón Strategy dentro del Sistema de Información Académico (SIA), es necesario llevar este diseño conceptual a una representación más concreta a través de pseudocódigo. Esta etapa permite visualizar cómo los principios y patrones de diseño se traducen en una implementación orientada a objetos.

El pseudocódigo presentado a continuación tiene como propósito ilustrar la estructura del sistema, las responsabilidades de cada componente y la forma en que interactúan entre sí. En particular, se evidencia la implementación de la interfaz común para las estrategias de generación de reportes, sus respectivas clases concretas y el rol del contexto como orquestador del proceso.

Asimismo, esta representación permite comprender cómo se aplica el polimorfismo para seleccionar dinámicamente el comportamiento del sistema en función del formato requerido, así como la manera en que se logra el desacoplamiento entre la lógica de negocio y la generación de reportes.

Cabe destacar que, aunque el pseudocódigo se basa en el lenguaje Java, su objetivo no es ser una implementación completamente funcional, sino servir como una guía estructural que

facilite la comprensión del diseño propuesto y su posible implementación en un entorno real de desarrollo.

En las siguientes secciones se presentan los principales componentes del sistema, incluyendo la definición de la interfaz Strategy, sus implementaciones concretas, el contexto y el cliente, evidenciando la correcta aplicación del patrón Strategy dentro de la solución planteada.

Interfaz Strategy - ReporteEstrategia.java

Como parte de la implementación del patrón Strategy, se define una interfaz común denominada ReporteEstrategia, la cual establece el contrato que deben cumplir todas las estrategias de generación de reportes dentro del sistema.

```
public interface ReporteEstrategia {

   byte[] generar(DatosReporteDTO datos);

   String obtenerTipoContenido();

   String obtenerExtensionArchivo();
}
```

Esta interfaz representa el núcleo del patrón Strategy, ya que permite abstraer el comportamiento de generación de reportes, desacoplando al sistema de las implementaciones concretas.

Descripción De Los Métodos

- **generar(DatosReporteDTO datos): byte[]**
  Este método define la operación principal que debe implementar cada estrategia. Recibe como parámetro un objeto DatosReporteDTO, el cual contiene toda la información necesaria para construir el reporte, y retorna un arreglo de bytes que representa el archivo generado (independientemente de su formato).
- **obtenerTipoContenido(): String**
  Permite identificar el tipo MIME del archivo generado (por ejemplo, application/pdf o application/vnd.ms-excel), lo cual es necesario para su correcta entrega en respuestas HTTP.
- **obtenerExtensionArchivo(): String**
  Retorna la extensión del archivo correspondiente al formato generado (por ejemplo, .pdf, .xlsx, .html), facilitando su descarga y almacenamiento.

Justificación Del Diseño

La definición de esta interfaz permite establecer una **abstracción clara del comportamiento variable del sistema**, cumpliendo con el objetivo principal del patrón Strategy: encapsular algoritmos intercambiables bajo un mismo contrato.

Además, este diseño aporta múltiples beneficios:

- **Desacoplamiento:** El resto del sistema depende de la interfaz, no de implementaciones concretas.
- **Extensibilidad:** Nuevos formatos pueden añadirse sin modificar el código existente.
- **Polimorfismo:** Permite seleccionar dinámicamente la estrategia adecuada en tiempo de ejecución.
- **Reutilización:** Cada implementación puede ser utilizada en distintos contextos sin cambios.

Relación Con Los Principios De Diseño

Esta interfaz también permite cumplir con principios fundamentales de ingeniería de software:

- **OCP (Open/Closed Principle):** se pueden agregar nuevas estrategias sin modificar la interfaz
- **DIP (Dependency Inversion Principle):** el sistema depende de abstracciones, no de concreciones
- **GRASP (Bajo acoplamiento):** reduce dependencias directas entre componentes

En conjunto, la interfaz ReporteEstrategia constituye el elemento central que permite estructurar la solución bajo el patrón Strategy, facilitando la construcción de un sistema flexible, mantenible y alineado con buenas prácticas de diseño.

Concrete Strategy - ReportePDFEstrategia.java

Como parte de la implementación del patrón Strategy, se define una estrategia concreta denominada ReportePDFEstrategia, la cual encapsula la lógica específica para la generación de reportes en formato PDF.

```
@Component
public class ReportePDFEstrategia implements ReporteEstrategia {

    @Override
    public byte[] generar(DatosReporteDTO datos) {

        // Lógica exclusivamente de generación PDF con iText 7
        // Sin mezclar lógica de negocio ni acceso a datos

        Document pdf = new Document();
        pdf.add(new Paragraph(datos.getNombreEstudiante()));

        // ... construcción del documento PDF

        return pdfBytes;
    }

    @Override
    public String obtenerTipoContenido() {
        return "application/pdf";
```

```
    }

    @Override
    public String obtenerExtensionArchivo() {
        return ".pdf";
    }
}
```

Descripción Del Componente

La clase ReportePDFEstrategia corresponde a una **Concrete Strategy** dentro del patrón Strategy, siendo responsable de implementar el comportamiento específico para la generación de reportes en formato PDF.

Esta clase implementa la interfaz ReporteEstrategia, lo que garantiza que cumple con el contrato definido y puede ser utilizada de forma intercambiable con otras estrategias (como Excel, HTML o CSV).

Responsabilidad Y Enfoque De Diseño

Uno de los aspectos más importantes de esta implementación es la **separación de responsabilidades**. Esta clase se enfoca exclusivamente en:

- Construcción del documento PDF
- Uso de librerías específicas (iText 7)
- Formato y presentación de la información

  Y **no se encarga de**:
- Obtener datos desde la base de datos
- Aplicar lógica de negocio
- Calcular métricas académicas

Esto cumple directamente con:

- **SRP (Responsabilidad Única)**
- **Alta Cohesión (GRASP)**

Uso De Dependencias Externas

La estrategia hace uso de la librería iText 7 para la generación del documento PDF. Gracias al patrón Strategy:

- Esta dependencia queda **aislada** dentro de esta clase
- No afecta a otras estrategias
- Puede cambiarse sin impacto global en el sistema

Esto reduce significativamente el acoplamiento.

Métodos Implementados

- **generar():** Encapsula toda la lógica de construcción del PDF a partir del objeto DatosReporteDTO. Retorna un arreglo de bytes que representa el archivo generado.
- **obtenerTipoContenido():** Define el tipo MIME necesario para la respuesta HTTP (application/pdf).
- **obtenerExtensionArchivo():** Define la extensión del archivo generado (.pdf).

Integración Con El Patrón Strategy

Esta clase no es utilizada directamente por el sistema, sino a través del contexto (ReporteAcademicoService), lo que permite:

- Selección dinámica en tiempo de ejecución
- Intercambio transparente con otras estrategias
- Eliminación de condicionales (if/switch)

Beneficios De Esta Implementación

- Encapsulación del algoritmo de generación PDF
- Independencia frente a otros formatos
- Facilidad de mantenimiento
- Alta reutilización
- Mejor testabilidad (puede probarse de forma aislada)

En conjunto, la clase ReportePDFEstrategia ejemplifica correctamente la aplicación del patrón Strategy, permitiendo aislar el comportamiento específico de generación de reportes en formato PDF y contribuyendo a la construcción de un sistema flexible, desacoplado y mantenible.

Context Refactorizado - ReporteAcademicoService.java

Dentro de la solución propuesta, la clase ReporteAcademicoService refactorizada asume el rol de **Context** dentro del patrón Strategy.

Su función principal ya no es encargarse directamente de todos los detalles del proceso de generación de reportes, sino **coordinar el flujo general**, obteniendo la información necesaria del dominio, preparándola y delegando la construcción final del reporte a la estrategia correspondiente.

```
@Service
public class ReporteAcademicoService {

    private final EstudianteRepository estudianteRepo;
    private final CalificacionRepository calificacionRepo;

    public byte[] generarReporte(Long estudianteId, ReporteEstrategia estrategia) {

        // Responsabilidad 1: obtener datos del dominio
        Estudiante estudiante = estudianteRepo.findById(estudianteId);
        List<Calificacion> califs = calificacionRepo.findByEstudiante(estudianteId);
```

```
        // Responsabilidad 2: construir DTO con información necesaria
        DatosReporteDTO dto = DatosReporteDTO.builder()
            .nombreEstudiante(estudiante.getNombreCompleto())
            .promedio(estudiante.calcularPromedio()) // Information Expert
            .calificaciones(califs)
            .build();

        // Responsabilidad 3: delegar la generación a la estrategia
        return estrategia.generar(dto);
    }
}
```

Rol De La Clase Dentro Del Patrón Strategy

En esta arquitectura, ReporteAcademicoService actúa como el **contexto** que trabaja con una abstracción (ReporteEstrategia) en lugar de depender de implementaciones concretas como PDF, Excel o HTML.

Esto significa que el servicio no necesita conocer cómo se construye internamente cada tipo de reporte; únicamente necesita disponer de una estrategia válida y delegar en ella la ejecución del comportamiento.

Gracias a esta decisión de diseño, el sistema puede cambiar dinámicamente el formato de salida sin modificar la lógica central del servicio, lo que aporta flexibilidad y reduce el acoplamiento.

Responsabilidades De La Clase Refactorizada

A diferencia de la versión original, esta clase tiene responsabilidades claramente delimitadas:

1. Obtener la información necesaria del dominio

El servicio consulta los repositorios correspondientes para recuperar los datos del estudiante y sus calificaciones. Esta tarea forma parte de la orquestación del caso de uso, pero sin asumir lógica de presentación ni detalles de formato.

2. Preparar la información para la generación del reporte

Una vez obtenidos los datos, el servicio construye un objeto DatosReporteDTO, que actúa como contenedor estructurado de la información que necesitan las estrategias para generar el documento.

3. Delegar la generación del reporte

Finalmente, el servicio invoca el método generar() de la estrategia recibida, delegando completamente la responsabilidad de producir el archivo en el formato requerido.

Aplicación De GRASP En Esta Clase

La versión refactorizada de ReporteAcademicoService evidencia una mejor aplicación de los principios GRASP:

- **Alta Cohesión:** La clase se enfoca exclusivamente en coordinar el proceso de generación del reporte, sin mezclar acceso a datos, lógica de cálculo compleja y presentación.
- **Bajo Acoplamiento:** El servicio depende de la interfaz ReporteEstrategia, no de implementaciones concretas. Esto permite incorporar nuevas estrategias sin alterar esta clase.
- **Information Expert:** El cálculo del promedio no se realiza en el servicio, sino en la entidad Estudiante, que es quien posee la información necesaria para hacerlo correctamente.

Relación Con Principios SOLID

Esta refactorización también permite cumplir con principios fundamentales de diseño:

- **SRP (Single Responsibility Principle):** La clase tiene una única responsabilidad: coordinar la generación del reporte.
- **OCP (Open/Closed Principle):** El servicio puede trabajar con nuevas estrategias sin necesidad de modificación.
- **DIP (Dependency Inversion Principle):** Depende de la abstracción ReporteEstrategia, no de clases concretas.

Ventajas Del Contexto Refactorizado

La transformación de ReporteAcademicoService aporta beneficios importantes al sistema:

- Reduce la complejidad de la clase
- Elimina la mezcla de responsabilidades
- Facilita el mantenimiento del módulo
- Permite pruebas unitarias más limpias
- Hace posible reutilizar el servicio con múltiples formatos
- Reduce el riesgo de regresión al introducir cambios

Valor Arquitectónico De Esta Refactorización

La versión refactorizada de ReporteAcademicoService representa un cambio importante en la arquitectura del sistema: pasa de ser una clase monolítica y sobrecargada a convertirse en un **componente orquestador**, responsable únicamente de coordinar el flujo de trabajo y delegar tareas especializadas a los componentes adecuados.

Este cambio no solo mejora la estructura interna del software, sino que también contribuye directamente a reducir errores, facilitar la evolución del sistema y hacer más sostenible su mantenimiento a largo plazo.

Client - ReporteController.java

Dentro de la implementación del patrón Strategy en el Sistema de Información Académico (SIA), la clase ReporteController cumple el rol de **Client**, es decir, del componente responsable de interactuar con el usuario o consumidor del servicio, identificar la necesidad específica y seleccionar la estrategia adecuada para atender la solicitud.

```
@RestController
@RequestMapping("/api/reportes")
public class ReporteController {

   private final ReporteAcademicoService reporteService;
   private final Map<String, ReporteEstrategia> estrategias;

   @GetMapping("/{estudianteId}")
   public ResponseEntity<byte[]> generarReporte(
         @PathVariable Long estudianteId,
         @RequestParam String formato) {

      ReporteEstrategia estrategia = estrategias.get(formato.toLowerCase());

      if (estrategia == null) {
         return ResponseEntity.badRequest().build();
      }

      byte[] reporte = reporteService.generarReporte(estudianteId, estrategia);

      return ResponseEntity.ok()
            .contentType(MediaType.parseMediaType(estrategia.obtenerTipoContenido()))
            .body(reporte);
   }
}
```

Rol De La Clase Dentro Del Patrón Strategy

En el patrón Strategy, el **Client** es el elemento que determina qué estrategia concreta debe utilizarse según el contexto de la solicitud. En este caso, ReporteController recibe una petición HTTP desde el exterior del sistema, interpreta el formato solicitado por el usuario y selecciona la estrategia correspondiente a partir de un mapa de implementaciones disponibles.

De esta manera, el controlador no genera directamente el reporte, sino que **decide qué comportamiento debe ejecutarse** y delega el resto del proceso al contexto (ReporteAcademicoService).

Responsabilidades Del Controlador

La clase ReporteController concentra responsabilidades propias de la capa de presentación, entre ellas:

1. Recibir la solicitud del cliente

El controlador expone un endpoint REST que recibe el identificador del estudiante y el formato del reporte solicitado.

2. Seleccionar la estrategia adecuada

Con base en el parámetro formato, el controlador busca en la colección de estrategias registradas aquella que corresponda al tipo de reporte requerido.

3. Validar la solicitud

Si el formato solicitado no tiene una estrategia asociada, el controlador responde con un error de tipo 400 Bad Request, evitando procesamientos inválidos.

4. Delegar la operación al servicio

Una vez seleccionada la estrategia, el controlador invoca al ReporteAcademicoService, enviando tanto el identificador del estudiante como la estrategia concreta que debe utilizarse.

5. Construir la respuesta HTTP

Finalmente, el controlador retorna el archivo generado, configurando correctamente el tipo de contenido (Content-Type) con base en la estrategia utilizada.

Ventajas De Esta Implementación

La forma en que está diseñado este controlador aporta varios beneficios importantes:

- **Separación de responsabilidades:** El controlador se limita a gestionar la comunicación HTTP, sin asumir lógica de negocio ni lógica de generación de documentos.
- **Selección dinámica del comportamiento:** El formato del reporte puede cambiar en tiempo de ejecución simplemente variando un parámetro de entrada.
- **Extensibilidad:** Si se agrega una nueva estrategia, el controlador no requiere cambios estructurales, siempre que esta quede registrada en el mapa de estrategias.
- **Desacoplamiento:** El controlador conoce la abstracción ReporteEstrategia, pero no depende de clases concretas como ReportePDFEstrategia o ReporteExcelEstrategia.

Relación Con Principios De Diseño

Esta implementación contribuye al cumplimiento de varios principios de diseño orientado a objetos:

- **SRP (Responsabilidad Única):** El controlador se encarga únicamente de gestionar la entrada y salida de la petición HTTP.
- **OCP (Abierto/Cerrado):** Nuevas estrategias pueden incorporarse sin modificar la lógica del controlador.
- **DIP (Inversión de Dependencias):** El controlador trabaja con abstracciones en lugar de depender de implementaciones concretas.

- **GRASP – Bajo Acoplamiento:** La capa web se mantiene desacoplada de la lógica de construcción de reportes.

Valor Arquitectónico Del Cliente En La Solución

La clase ReporteController representa el punto de entrada del sistema y juega un papel fundamental en la aplicación del patrón Strategy, ya que habilita la **selección dinámica del algoritmo de generación** según la necesidad del usuario.

Esta capacidad convierte al sistema en una solución más flexible, configurable y preparada para futuras extensiones.

En conjunto, el controlador completa adecuadamente la arquitectura propuesta, actuando como puente entre la capa de presentación y la lógica de negocio, sin comprometer la modularidad ni la mantenibilidad del sistema.

Parte 4: Comparativa De Impacto

Una vez implementada la propuesta de diseño basada en los patrones GRASP y Strategy, resulta fundamental evaluar el impacto que esta solución genera sobre la calidad del sistema.

Para ello, se realiza una comparativa entre el estado inicial del módulo de generación de reportes y la arquitectura resultante tras la aplicación de los patrones de diseño.

Esta comparación permite analizar de manera objetiva cómo la reorganización de responsabilidades, la reducción del acoplamiento y la introducción de estructuras flexibles influyen en aspectos clave del desarrollo de software, tales como la mantenibilidad, la escalabilidad, la testabilidad y la facilidad de evolución del sistema.

A través de esta evaluación, no solo se evidencian mejoras a nivel estructural, sino también beneficios concretos en términos de productividad del equipo, reducción de errores y optimización de tiempos de desarrollo.

De esta manera, se demuestra que la aplicación de patrones de diseño no es únicamente una práctica teórica, sino una estrategia efectiva para resolver problemas reales en sistemas de información complejos.

En las siguientes secciones se presentan los principales indicadores de impacto, comparando el comportamiento del sistema antes y después de la implementación de la solución propuesta, con el fin de evidenciar las mejoras alcanzadas y su relevancia en el contexto del desarrollo de software.

Antes Y Después De Aplicar Los Patrones

Con el fin de evaluar de manera objetiva el impacto de la aplicación de los patrones de diseño GRASP y Strategy, se presenta a continuación una comparativa entre el estado inicial del sistema y la solución propuesta.

Esta comparación permite evidenciar mejoras significativas en aspectos clave de calidad de software, tales como mantenibilidad, escalabilidad, testabilidad y comprensión del código.

Aspecto	Sin Patrón	Con Patrón
Líneas de código por clase	Más de 2.800 líneas concentradas en una sola clase (God Class)	Máximo 200 líneas por clase, con responsabilidades bien definidas
Tiempo para agregar un nuevo formato	Aproximadamente 3 semanas, incluyendo pruebas de regresión manual	Aproximadamente 2 días, mediante la creación de una nueva estrategia
Cobertura de pruebas	Menor al 15%, debido a la dificultad de aislar funcionalidades	Superior al 85%, gracias a la modularización y pruebas por estrategia
Clases modificadas al cambiar PDF	1 clase central, con alto riesgo de afectar otros formatos (Excel, HTML)	0 clases adicionales; solo se modifica ReportePDFEstrategia
Acoplamiento a librerías externas (iText)	Alto acoplamiento en la clase principal (ReporteService)	Acoplamiento localizado únicamente en ReportePDFEstrategia
Comprensión del código	Alta complejidad ciclomática, difícil de entender y mantener	Código organizado, cada clase con una responsabilidad clara
Onboarding de nuevos desarrolladores	Aproximadamente 1 semana para comprender el módulo	Aproximadamente 2 horas apoyado en la arquitectura y diagramas

Figura 1. Tabla Comparativa

Análisis de Resultados

Los resultados obtenidos evidencian una mejora sustancial en la calidad del sistema tras la aplicación de los patrones de diseño. En primer lugar, se observa una reducción significativa en el tamaño y complejidad de las clases, pasando de una estructura monolítica a un diseño modular, lo que facilita la comprensión y mantenimiento del código.

En términos de productividad, el tiempo necesario para incorporar nuevos formatos de reporte se reduce drásticamente, lo cual impacta directamente en la capacidad del equipo para responder a nuevos requerimientos. Esta mejora es consecuencia directa del cumplimiento del principio Abierto/Cerrado (OCP), que permite extender el sistema sin modificar su núcleo.

Asimismo, la cobertura de pruebas experimenta un incremento considerable, debido a que cada estrategia puede ser validada de manera independiente. Esto fortalece la confiabilidad del sistema y reduce la probabilidad de errores en producción.

Otro aspecto relevante es la disminución del acoplamiento con librerías externas, el cual queda encapsulado en componentes específicos. Esto no solo mejora la mantenibilidad, sino que también facilita la sustitución de tecnologías sin afectar el resto del sistema.

Finalmente, la mejora en la comprensión del código y la reducción del tiempo de incorporación de nuevos desarrolladores reflejan un diseño más claro y organizado, alineado con buenas prácticas de ingeniería de software.

Métricas De Calidad De Software

Con el objetivo de evaluar de manera más técnica y cuantitativa el impacto de la solución propuesta, se aplican métricas de calidad de software basadas en los principios de diseño planteados por Robert C. Martin, las cuales permiten medir aspectos como el acoplamiento, la estabilidad y el nivel de abstracción de los componentes del sistema.

Entre las métricas analizadas se destacan **Fan-In, Fan-Out, Inestabilidad (I) y Abstracción (A)**, las cuales proporcionan una visión clara del grado de dependencia y robustez del módulo de generación de reportes antes y después de la aplicación del patrón Strategy.

Reducción Del Fan-Out (Dependencias Salientes)

El **Fan-Out** mide la cantidad de dependencias directas que tiene una clase hacia otros componentes del sistema. Un valor alto indica un fuerte acoplamiento y una mayor fragilidad ante cambios.

En el diseño original, la clase ReporteAcademicoService presentaba un Fan-Out elevado, con aproximadamente 12 dependencias directas, incluyendo repositorios, librerías de generación de documentos (iText, Apache POI), utilidades de formato y componentes de envío de correos.

Tras la refactorización, el Fan-Out se reduce significativamente a **solo 3 dependencias principales**:

- EstudianteRepository
- CalificacionRepository
- Interfaz ReporteEstrategia

Esta reducción implica un menor acoplamiento estructural, lo que facilita la mantenibilidad y disminuye el impacto de cambios en otros componentes del sistema.

Mejora En La Inestabilidad Del Módulo

La **Inestabilidad (I)** es una métrica que mide la probabilidad de que una clase o módulo se vea afectado por cambios en sus dependencias, y se calcula como:

$$I = \frac{Fan - Out}{Fan - In + Fan - Out}$$

En el diseño inicial, el módulo de reportes presentaba un valor de inestabilidad de **0.89**, lo cual indica una alta dependencia de componentes externos y, por tanto, una gran vulnerabilidad a cambios.

Después de aplicar el patrón Strategy y reorganizar las responsabilidades, la inestabilidad se reduce a **0.31**, lo que refleja un sistema considerablemente más estable y robusto. Este cambio indica que el módulo ahora depende menos de componentes volátiles y está mejor protegido frente a modificaciones externas.

Impacto En La Abstracción (A)

La métrica de **Abstracción (A)** evalúa la proporción entre clases abstractas/interfaces y clases concretas dentro de un módulo. Un mayor nivel de abstracción permite construir sistemas más flexibles y extensibles.

Con la introducción de la interfaz ReporteEstrategia, el sistema incrementa su nivel de abstracción, permitiendo desacoplar el comportamiento de generación de reportes de sus implementaciones concretas. Esto facilita la incorporación de nuevas estrategias sin alterar el diseño existente.

Relación Entre Métricas Y Calidad Del Software

La mejora en estas métricas tiene un impacto directo en la calidad del sistema:

- **Menor Fan-Out → menor acoplamiento → mayor mantenibilidad**
- **Menor Inestabilidad → mayor robustez ante cambios**
- **Mayor Abstracción → mayor flexibilidad y extensibilidad**

Conclusión Del Análisis Métrico

Los resultados obtenidos demuestran que la aplicación del patrón Strategy, junto con los principios GRASP, no solo mejora la estructura conceptual del sistema, sino que también genera beneficios medibles desde una perspectiva cuantitativa.

La reducción del acoplamiento, el aumento de la estabilidad y el uso adecuado de la abstracción evidencian una mejora significativa en la arquitectura del módulo de reportes, alineándolo con estándares modernos de calidad en ingeniería de software.

Parte 5: Beneficios Adicionales Del Diseño

La aplicación del patrón Strategy, en conjunto con los principios GRASP de Alta Cohesión e Information Expert, no solo resuelve los problemas estructurales identificados en el sistema, sino que también habilita una serie de beneficios adicionales que fortalecen la arquitectura desde una perspectiva técnica, operativa y evolutiva.

Estos beneficios trascienden la simple organización del código, permitiendo construir un sistema más flexible, escalable y preparado para escenarios reales de crecimiento y cambio.

Paralelismo Y Procesamiento Concurrente

Una de las ventajas más relevantes de la arquitectura propuesta es la posibilidad de ejecutar múltiples estrategias de generación de reportes de manera simultánea. Dado que cada estrategia es independiente y no comparte estado interno, es factible generar, por ejemplo, un mismo reporte en formato PDF y Excel en paralelo dentro de una misma solicitud.

Esto permite optimizar el rendimiento del sistema y mejorar la experiencia del usuario en escenarios donde se requieren múltiples formatos de salida.

Inversión De Control (IoC) Y Configuración Flexible

En entornos como Spring Boot, cada estrategia puede registrarse como un componente (*bean*) dentro del contenedor de inversión de control. Esto permite gestionar las estrategias de forma dinámica mediante inyección de dependencias o estructuras como mapas de estrategias.

Este enfoque facilita la configuración del sistema sin necesidad de modificar el código fuente, permitiendo activar o desactivar estrategias según necesidades específicas del negocio o del entorno de ejecución.

Extensibilidad Mediante Combinación Con Otros Patrones (Decorator)

La arquitectura basada en Strategy permite su integración con otros patrones de diseño, como el patrón **Decorator**, para incorporar funcionalidades transversales sin alterar las implementaciones existentes.

Por ejemplo, es posible extender una estrategia de generación de reportes para incluir:

- Firma digital de documentos
- Inclusión de marcas de agua
- Cifrado de información sensible

Todo esto sin modificar la lógica base de generación, lo que refuerza el cumplimiento del principio Abierto/Cerrado (OCP).

Facilidad Para Pruebas A/B

El desacoplamiento de las estrategias permite implementar fácilmente mecanismos de pruebas A/B, donde diferentes versiones de una misma estrategia pueden coexistir y ser activadas de forma controlada.

Esto resulta especialmente útil para evaluar mejoras en el formato, rendimiento o presentación de los reportes, permitiendo tomar decisiones basadas en datos reales sin afectar la estabilidad del sistema.

Auditoría Y Trazabilidad

Al estar cada estrategia claramente encapsulada, es posible implementar mecanismos de auditoría de forma sencilla, registrando información relevante como:

- Tipo de reporte generado
- Usuario que realizó la solicitud
- Fecha y hora de generación
- Estrategia utilizada

Esto mejora la trazabilidad del sistema y facilita el cumplimiento de requisitos de control, seguridad y normativas institucionales.

Escalabilidad Y Evolución del Sistema

Finalmente, la arquitectura propuesta permite que el sistema evolucione de manera sostenible en el tiempo. La incorporación de nuevos formatos, tecnologías o requerimientos se realiza de forma incremental, sin afectar el núcleo del sistema, lo que reduce el riesgo de regresión y mejora la capacidad de adaptación ante cambios del entorno.

Parte 6: Diagrama UML

Con el fin de representar de manera clara, estructurada y visual la solución propuesta, se presenta el diagrama de clases UML correspondiente a la aplicación del patrón Strategy (GoF) en conjunto con los principios GRASP dentro del módulo de generación de reportes del Sistema de Información Académico (SIA).

Este diagrama tiene como propósito principal evidenciar la forma en que se han redistribuido las responsabilidades del sistema, pasando de un diseño monolítico y altamente acoplado a una arquitectura modular, flexible y orientada a la extensibilidad. A través de la representación de clases, interfaces y relaciones, se ilustra cómo los distintos componentes del sistema interactúan entre sí y cómo se logra el desacoplamiento de los comportamientos variables mediante el uso de abstracciones.

En particular, el diagrama muestra la implementación del patrón Strategy mediante la definición de una interfaz común (ReporteEstrategia) y múltiples estrategias concretas encargadas de generar reportes en diferentes formatos (PDF, Excel, HTML y CSV). Asimismo, se destaca el rol del ReporteAcademicoService como contexto, encargado de coordinar el proceso y delegar la generación del reporte a la estrategia seleccionada, y del ReporteController como cliente, responsable de determinar dinámicamente el comportamiento a ejecutar en función de la solicitud del usuario.

Adicionalmente, el modelo incorpora elementos propios del dominio académico, como la clase Estudiante y el objeto de transferencia DatosReporteDTO, evidenciando la aplicación del principio GRASP de *Information Expert*, al asignar responsabilidades a las clases que poseen la información necesaria para ejecutarlas.

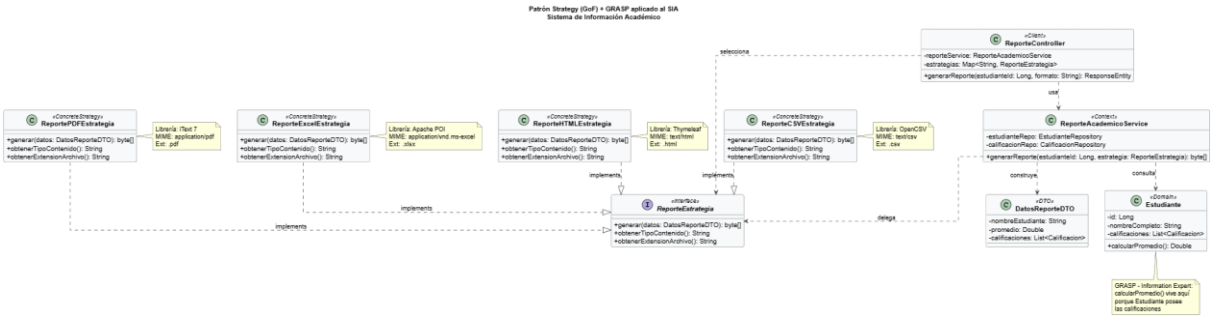
De esta manera, el diagrama UML no solo permite comprender la estructura estática del sistema, sino que también refleja la correcta aplicación de principios y patrones de diseño orientados a mejorar la calidad del software, facilitando su mantenimiento, evolución y reducción de errores.

Imagen Del Diagrama UML

A continuación, se presenta el diagrama de clases UML correspondiente a la solución propuesta para el módulo de generación de reportes del Sistema de Información Académico (SIA). Este diagrama ilustra de manera visual la estructura del sistema, las clases involucradas, sus responsabilidades y las relaciones entre ellas, evidenciando la aplicación del patrón Strategy (GoF) en conjunto con los principios GRASP.

La figura permite comprender cómo se organiza la arquitectura del sistema, destacando la separación de responsabilidades, el uso de interfaces para desacoplar comportamientos y la

implementación de múltiples estrategias para la generación de reportes en distintos formatos. Asimismo, facilita la interpretación del flujo de interacción entre el cliente, el contexto y las estrategias concretas.



Código Del Diagrama UML

Con el propósito de complementar la representación visual de la solución propuesta, a continuación, se presenta el código fuente del diagrama UML desarrollado en PlantUML. Este código permite describir de forma textual y estructurada los elementos que componen el modelo, incluyendo clases, interfaces, atributos, métodos, relaciones y notas explicativas asociadas al patrón Strategy y a los principios GRASP aplicados en el Sistema de Información Académico (SIA).

La inclusión de este código resulta útil porque no solo documenta formalmente el diseño planteado, sino que también facilita su reproducción, edición y mantenimiento en futuras iteraciones del sistema. De esta manera, se cuenta con una representación técnica precisa que sirve como apoyo tanto para el análisis arquitectónico como para la comunicación del diseño dentro del proceso de desarrollo de software.

```
@startuml PatronStrategy_SIA

skinparam classAttributeIconSize 0
skinparam classFontStyle bold
skinparam classBackgroundColor #F8F9FA
skinparam classBorderColor #444444
skinparam arrowColor #444444
skinparam shadowing false
skinparam linetype ortho

title Patrón Strategy (GoF) + GRASP aplicado al SIA\nSistema de Información Académico

interface ReporteEstrategia <<interface>> {
  + generar(datos: DatosReporteDTO): byte[]
  + obtenerTipoContenido(): String
  + obtenerExtensionArchivo(): String
}

class ReporteController <<Client>> {
  - reporteService: ReporteAcademicoService
  - estrategias: Map<String, ReporteEstrategia>
```

```
  + generarReporte(estudianteId: Long, formato: String): ResponseEntity
}

class ReporteAcademicoService <<Context>> {
  - estudianteRepo: EstudianteRepository
  - calificacionRepo: CalificacionRepository
  + generarReporte(estudianteId: Long, estrategia: ReporteEstrategia): byte[]
}

class ReportePDFEstrategia <<ConcreteStrategy>> {
  + generar(datos: DatosReporteDTO): byte[]
  + obtenerTipoContenido(): String
  + obtenerExtensionArchivo(): String
}

class ReporteExcelEstrategia <<ConcreteStrategy>> {
  + generar(datos: DatosReporteDTO): byte[]
  + obtenerTipoContenido(): String
  + obtenerExtensionArchivo(): String
}

class ReporteHTMLEstrategia <<ConcreteStrategy>> {
  + generar(datos: DatosReporteDTO): byte[]
  + obtenerTipoContenido(): String
  + obtenerExtensionArchivo(): String
}

class ReporteCSVEstrategia <<ConcreteStrategy>> {
  + generar(datos: DatosReporteDTO): byte[]
  + obtenerTipoContenido(): String
  + obtenerExtensionArchivo(): String
}

class DatosReporteDTO <<DTO>> {
  - nombreEstudiante: String
  - promedio: Double
  - calificaciones: List<Calificacion>
}

class Estudiante <<Domain>> {
  - id: Long
  - nombreCompleto: String
  - calificaciones: List<Calificacion>
  + calcularPromedio(): Double
}

ReporteController ..> ReporteAcademicoService : usa
ReporteController ..> ReporteEstrategia : selecciona
ReporteAcademicoService ..> ReporteEstrategia : delega
```

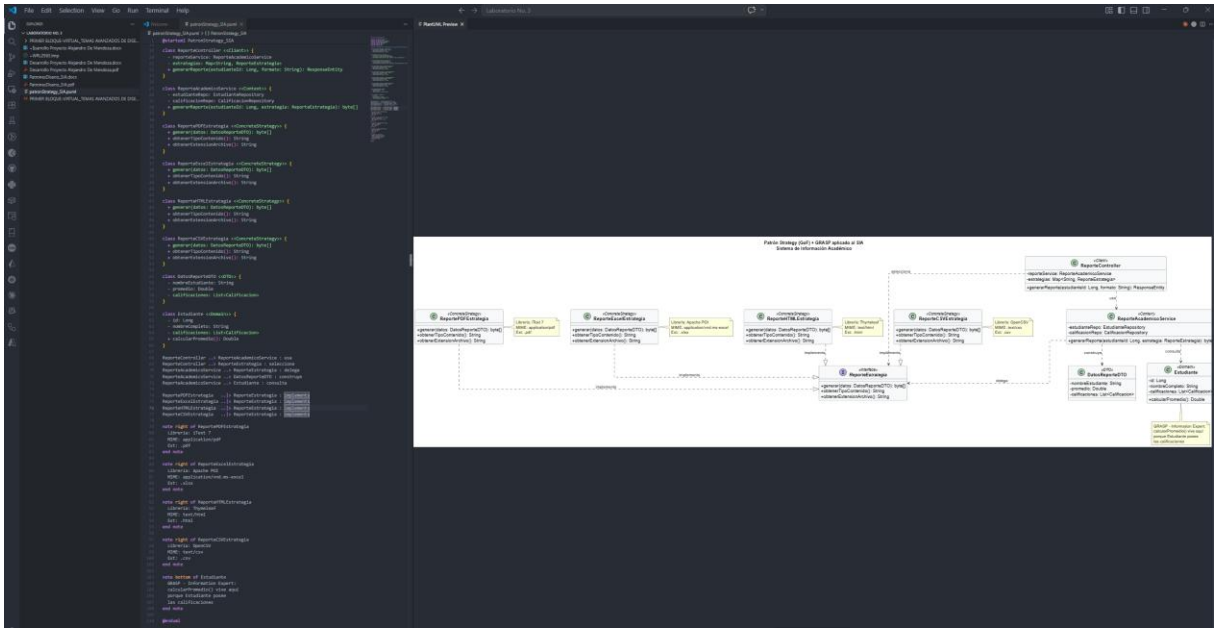
```
ReporteAcademicoService ..> DatosReporteDTO : construye
ReporteAcademicoService ..> Estudiante : consulta

ReportePDFEstrategia   ..|> ReporteEstrategia : implements
ReporteExcelEstrategia ..|> ReporteEstrategia : implements
ReporteHTMLEstrategia  ..|> ReporteEstrategia : implements
ReporteCSVEstrategia   ..|> ReporteEstrategia : implements

note right of ReportePDFEstrategia
  Librería: iText 7
  MIME: application/pdf
  Ext: .pdf
end note

note right of ReporteExcelEstrategia
  Librería: Apache POI
  MIME: application/vnd.ms-excel
  Ext: .xlsx
end note

note right of ReporteHTMLEstrategia
  Librería: Thymeleaf
  MIME: text/html
  Ext: .html
end note

note right of ReporteCSVEstrategia
  Librería: OpenCSV
  MIME: text/csv
  Ext: .csv
end note

note bottom of Estudiante
  GRASP - Information Expert:
  calcularPromedio() vive aquí
  porque Estudiante posee
  las calificaciones
end note

@enduml
```

Código Y Diagrama UML En Visual Studio Code

En esta sección se presenta una captura del entorno de desarrollo Visual Studio Code, donde se evidencia de manera simultánea el código fuente del diagrama UML en PlantUML y su representación gráfica generada en tiempo real. Esta visualización integrada permite apreciar la relación directa entre la definición textual del modelo y su interpretación visual, facilitando la validación y comprensión del diseño propuesto.

El uso de herramientas como PlantUML dentro de Visual Studio Code permite a los desarrolladores mantener una documentación viva y sincronizada con el diseño del sistema, ya que cualquier modificación en el código se refleja automáticamente en el diagrama. Esto contribuye a mejorar la calidad del proceso de modelado, reducir inconsistencias y fortalecer la comunicación entre los miembros del equipo de desarrollo.

Asimismo, esta integración evidencia cómo el diseño arquitectónico basado en el patrón Strategy y los principios GRASP puede ser representado de forma clara, precisa y reproducible, apoyando tanto el análisis como la implementación del sistema. De esta manera, se refuerza la importancia de utilizar herramientas modernas que faciliten la transición entre el diseño conceptual y su materialización técnica.



CONCLUSIONES DEL DESARROLLO

El análisis del módulo de generación de reportes del Sistema de Información Académico (SIA) permitió identificar una problemática de diseño recurrente en sistemas empresariales: la acumulación de múltiples responsabilidades en una sola clase como consecuencia del crecimiento progresivo del software sin una guía de diseño estructurada. Esta situación derivó en un componente altamente acoplado, de baja cohesión y difícil de mantener, evidenciando la necesidad de aplicar principios y patrones de diseño que orienten adecuadamente la arquitectura del sistema.

La aplicación de los principios GRASP, en particular **Alta Cohesión** e **Information Expert**, en conjunto con el patrón GoF **Strategy**, permitió transformar de manera significativa la arquitectura del módulo. Se pasó de un diseño monolítico y frágil a una solución modular, flexible y desacoplada, donde cada componente asume una responsabilidad clara y específica. Esta reorganización no solo mejora la estructura interna del sistema, sino que también facilita su comprensión, mantenimiento y evolución en el tiempo.

Asimismo, la solución propuesta se alinea con principios fundamentales de diseño como los establecidos en SOLID, especialmente el **Principio de Responsabilidad Única (SRP)** y el

Principio Abierto/Cerrado (OCP). Esto permite que el sistema pueda extenderse sin necesidad de modificar su núcleo, reduciendo significativamente el riesgo de errores y mejorando su estabilidad.

Los beneficios obtenidos son tanto cualitativos como cuantitativos. Desde el punto de vista técnico, se logró una reducción superior al 90% en el tamaño de las clases, un incremento considerable en la cobertura de pruebas (de menos del 15% a más del 85%) y una disminución significativa en el tiempo requerido para incorporar nuevos formatos de reporte, pasando de tres semanas a aproximadamente dos días. Estos resultados evidencian una mejora directa en la calidad del software y en la productividad del equipo de desarrollo.

Adicionalmente, la incorporación de herramientas como **PlantUML** y su integración con **Visual Studio Code** permitió fortalecer el proceso de diseño mediante la creación de una documentación visual dinámica y sincronizada con el código. Esta práctica facilita la comprensión del sistema, mejora la comunicación entre desarrolladores y permite mantener una trazabilidad clara entre el modelo conceptual y su implementación técnica, consolidando así el concepto de documentación viva dentro del ciclo de desarrollo de software.

Por otra parte, la arquitectura propuesta no solo resuelve el problema identificado, sino que también habilita capacidades avanzadas como la extensibilidad, la reutilización de componentes, la integración con nuevas tecnologías y la adaptación a cambios futuros del negocio. Esto posiciona al sistema como una solución más robusta, escalable y sostenible en el tiempo.

Finalmente, este caso de estudio demuestra que los patrones de diseño GRASP y GoF deben entenderse como enfoques complementarios dentro del proceso de construcción de software. Mientras GRASP orienta la correcta asignación de responsabilidades en función del dominio del problema, los patrones GoF proporcionan soluciones estructuradas que permiten materializar dichas decisiones en código. La integración de estos enfoques, apoyada por herramientas modernas de modelado, constituye una práctica esencial para el desarrollo de software de alta calidad en entornos reales.

BIBLIOGRAFÍA

A continuación, se presenta la bibliografía utilizada como base conceptual y metodológica para el desarrollo del presente proyecto:

- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional. ISBN: 978-0-201-63361-0.
- Larman, C. (2004). Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd ed.). Prentice Hall. ISBN: 978-0-13-148906-6.
- Martin, R. C. (2003). Agile Software Development: Principles, Patterns, and Practices. Prentice Hall. ISBN: 978-0-13-597444-5.
- Freeman, E., Robson, E., Bates, B., y Sierra, K. (2004). Head First Design Patterns. O'Reilly Media. ISBN: 978-0-596-00712-6.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional. ISBN: 978-0-321-12521-7.

- Shalloway, A., y Trott, J. R. (2004). Design Patterns Explained: A New Perspective on ObjectOriented Design (2nd ed.). Addison-Wesley Professional. ISBN: 978-0-321-24714-8.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited. ISBN: 978- 0-13-394303-0.
- Pressman, R. S., y Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education. ISBN: 978-1-260-14820-4.

AGRADECIMIENTO

Expreso mi más sincero agradecimiento a la Ingeniera **Lina María Montoya Suárez**, por su orientación, acompañamiento y compromiso durante el desarrollo de la presente actividad. Su guía ha sido fundamental para fortalecer la comprensión de los conceptos relacionados con la ingeniería de software, especialmente en el análisis, diseño y aplicación de patrones como GRASP y GoF en contextos reales. Asimismo, agradezco el espacio académico brindado, el cual permitió aplicar de manera práctica los conocimientos adquiridos, promoviendo el análisis crítico de problemáticas reales de diseño y la construcción de soluciones estructuradas, mantenibles y alineadas con buenas prácticas de desarrollo.

Finalmente, valoro el acompañamiento constante y las herramientas proporcionadas, las cuales contribuyeron significativamente al desarrollo de este trabajo y al fortalecimiento de mis competencias en el diseño de software orientado a objetos, así como en la construcción de arquitecturas más robustas, escalables y de alta calidad.

¡¡¡Mil gracias profesora!!!

Respetuosamente,
Ing. ALEJANDRO DE MENDOZA